

Laporan Proyek 4: Pelaporan Eksperimen dan Kualitas Model

Informasi Proyek

- **Nama Proyek:** MLOps Pipeline untuk Analisis Sentimen Media Sosial (Twitter/X)
 - **Tanggal:** 27 November 2025
 - **Model:** BERTopic dengan IndoBERT Embeddings
 - **Domain:** Analisis Topik Tweet tentang Pemerintah Indonesia
-

Daftar Isi

1. Ringkasan Eksekutif
 2. Data Preprocessing
 3. Proses Eksperimen Training
 4. Evaluasi Model
 5. MLOps Tooling & Reproducibility
 6. Kesimpulan dan Rekomendasi
-

1. Ringkasan Eksekutif

Bab ini menyajikan gambaran umum mengenai proyek MLOps yang telah dikembangkan untuk melakukan analisis topik terhadap data tweet berbahasa Indonesia. Ringkasan eksekutif mencakup tujuan proyek, arsitektur sistem secara keseluruhan, serta teknologi yang digunakan dalam implementasi. Pemahaman yang baik terhadap bagian ini akan memberikan konteks yang diperlukan untuk memahami detail teknis pada bab-bab selanjutnya.

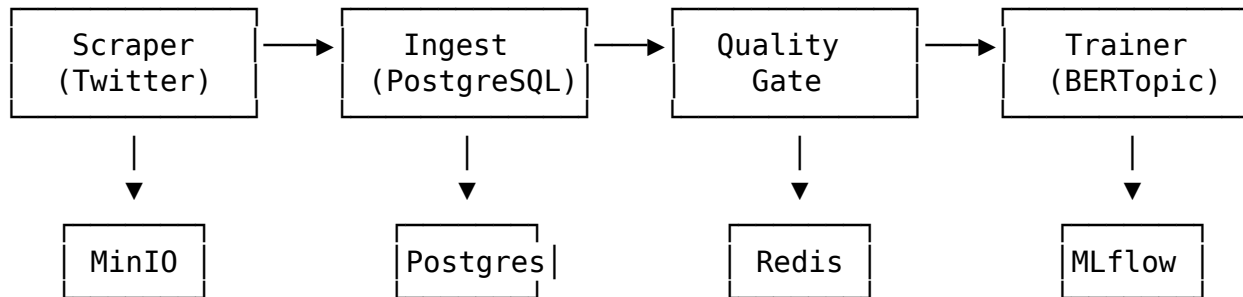
1.1 Tujuan Proyek

Proyek ini bertujuan untuk membangun sistem MLOps (Machine Learning Operations) yang komprehensif dan dapat diandalkan untuk menganalisis sentimen dan topik yang berkembang di media sosial, khususnya Twitter/X, terkait pemerintah Indonesia. Sistem ini dirancang untuk berjalan secara otomatis dan berkelanjutan, mulai dari pengumpulan data hingga pelatihan model dan monitoring performa. Fokus utama pengembangan meliputi: - **Pengumpulan data otomatis** dari Twitter/X - **Preprocessing dan validasi kualitas data** - **Training model topic modeling** menggunakan BERTopic - **Tracking eksperimen** menggunakan MLflow - **Deteksi drift** untuk monitoring model di production

1.2 Arsitektur Pipeline

Arsitektur pipeline MLOps yang dikembangkan mengikuti prinsip modularitas dan loose coupling, di mana setiap komponen memiliki tanggung jawab spesifik dan berkomunikasi melalui interface yang terdefinisi dengan jelas. Pendekatan ini memungkinkan

pengembangan, pengujian, dan deployment yang independen untuk setiap komponen. Pipeline terdiri dari empat tahap utama yang dieksekusi secara berurutan, dengan dukungan penyimpanan data terpusat menggunakan teknologi cloud-native.



1.3 Teknologi Stack

Pemilihan teknologi dalam proyek ini didasarkan pada beberapa pertimbangan utama: skalabilitas, kemudahan integrasi, dukungan komunitas yang kuat, serta kesesuaian dengan best practice industri untuk sistem MLOps. Kombinasi teknologi yang dipilih memungkinkan pipeline untuk berjalan secara reliable dalam environment containerized dan dapat di-scale sesuai kebutuhan workload.

Komponen	Teknologi
Data Collection	Twikit (Twitter API)
Object Storage	MinIO
Database	PostgreSQL
Cache	Redis
ML Model	BERTopic + IndoBERT
Experiment Tracking	MLflow
Orchestration	Apache Airflow
Containerization	Docker

2. Data Preprocessing

Data preprocessing merupakan tahap krusial dalam pipeline machine learning yang menentukan kualitas input untuk model. Pada bab ini akan dijelaskan secara detail proses pengumpulan data dari Twitter, pembersihan dan transformasi data, serta validasi kualitas yang dilakukan sebelum data digunakan untuk training. Setiap tahap preprocessing dirancang untuk memastikan data yang dihasilkan bersih, konsisten, dan representatif terhadap fenomena yang ingin dianalisis.

2.1 Proses Pengumpulan Data (Scraper)

Scraper bertanggung jawab untuk mengumpulkan data tweet secara otomatis dari platform Twitter/X. Komponen ini diimplementasikan dengan memperhatikan rate limiting

dan kebijakan anti-bot dari Twitter untuk memastikan pengumpulan data yang berkelanjutan tanpa risiko pemblokiran akun. Scraper menggunakan library Twikit yang menyediakan interface untuk mengakses Twitter API secara terprogram.

2.1.1 Konfigurasi Scraper Konfigurasi scraper dirancang untuk menyeimbangkan antara volume data yang dikumpulkan dengan keamanan akses terhadap API Twitter. Parameter delay dan rate limiting yang konservatif memastikan behavior scraper menyerupai pengguna manusia normal, sehingga mengurangi risiko deteksi sebagai bot oleh sistem keamanan Twitter.

```
# Query pencarian
SEARCH_QUERY = "pemerintah lang:id -filter:retweets"

# Anti-bot configuration
DELAY_MIN = 5.0          # Minimum delay antar request (detik)
DELAY_MAX = 12.0         # Maximum delay antar request (detik)
MAX_REQUESTS_PER_HOUR = 30
MAX_REQUESTS_PER_DAY = 200
```

2.1.2 Ekstraksi Fitur dari Tweet Setiap tweet yang berhasil dikumpulkan akan melalui proses ekstraksi fitur yang komprehensif. Proses ini mengubah data mentah dari API Twitter menjadi format terstruktur yang siap untuk analisis lebih lanjut. Fitur-fitur yang diekstrak mencakup informasi identifikasi, temporal, konten, user profile, engagement metrics, dan entitas yang terkandung dalam tweet. Berikut adalah detail kategori fitur yang diekstrak:

Kategori	Fitur yang Diekstrak
Identifikasi	tweet_id, content_hash, session_id
Temporal	created_at, collected_at
Konten	text, text_length, lang, possibly_sensitive
User	user_id, username, user_name, user_verified, user_followers, user_following
Engagement	retweet_count, like_count, reply_count, quote_count, view_count
Entitas	hashtags, mentions, urls, media_urls, cashtags
Flags	is_retweet, is_reply, is_quote, has_media, has_urls

2.1.3 Deduplikasi Deduplikasi merupakan langkah penting untuk memastikan tidak ada data redundan yang masuk ke dalam dataset. Sistem ini mengimplementasikan strategi **dual deduplication** yang menggunakan Redis sebagai penyimpanan key-value yang cepat. Pendekatan ganda ini memastikan bahwa baik tweet dengan ID yang sama maupun tweet dengan konten identik (meskipun berbeda ID, misalnya karena re-post) dapat terdeteksi dan difilter. 1. **ID-based**: Menyimpan tweet_id yang sudah diproses 2. **Content-based**: Menyimpan hash MD5 dari teks yang dinormalisasi

```
def generate_tweet_hash(text: str) -> str:
    """Generate content hash for deduplication."""
```

```

normalized = text.lower()
normalized = re.sub(r'@\w+', '', normalized)      # Remove mentions
normalized = re.sub(r'https?://\S+', '', normalized) # Remove URLs
normalized = ' '.join(normalized.split())
return hashlib.md5(normalized.encode('utf-8')).hexdigest()

```

2.2 Proses Ingest (Data Cleaning)

Setelah data dikumpulkan oleh scraper, tahap ingest bertanggung jawab untuk membersihkan dan menyimpan data ke dalam database PostgreSQL. Proses ini mencakup validasi data, pembersihan teks, dan ekstraksi fitur tambahan yang akan digunakan untuk analisis. Komponen ingest memastikan bahwa hanya data berkualitas tinggi yang masuk ke dalam sistem penyimpanan permanen.

2.2.1 Validasi Tweet Validasi tweet dilakukan untuk memfilter data yang tidak memenuhi kriteria kualitas minimum. Setiap tweet harus memiliki field wajib yang lengkap, panjang teks yang memadai, bahasa yang sesuai target (Indonesia atau Inggris), dan nilai engagement yang valid. Data yang tidak lolos validasi akan ditolak dan dicatat untuk keperluan debugging.

```

def validate_tweet(tweet: Dict[str, Any]):
    """Validate tweet data quality."""
    # Required fields
    required_fields = ['tweet_id', 'text', 'created_at', 'user_id']

    # Text length validation
    if len(text) < 10: return False, "Text too short"
    if len(text) > 5000: return False, "Text too long"

    # Language validation
    valid_languages = ['id', 'in', 'en'] # Indonesian & English
    if tweet['lang'] not in valid_languages: return False

    # Engagement validation (no negative values)
    if any(tweet[field] < 0 for field in ['retweet_count', 'like_count', 'reply_count']):
        return False

    return True, None

```

2.2.2 Text Cleaning Pipeline Pembersihan teks (text cleaning) merupakan tahap preprocessing yang kritis untuk mempersiapkan data tekstual sebelum diproses oleh model NLP. Pipeline pembersihan ini menangani berbagai masalah umum pada data teks dari media sosial seperti karakter HTML yang ter-encode, karakter zero-width yang tidak terlihat, dan normalisasi whitespace. Proses ini memastikan konsistensi format teks tanpa menghilangkan informasi semantik yang penting.

```

def clean_text(text: str) -> str:
    """Clean and normalize tweet text."""

```

```
# 1. Decode HTML entities
text = html.unescape(text)

# 2. Remove zero-width characters
text = re.sub(r'[\u200b\u200c\u200d\u200e\u200f\u2010\u2011\u2012\u2013\u2014\u2015\u2016\u2017\u2018\u2019\u201a\u201b\u201c\u201d\u201e\u201f\u2020\u2021\u2022\u2023\u2024\u2025\u2026\u2027\u2028\u2029\u202a\u202b\u202c\u202d\u202e\u202f\u2030\u2031\u2032\u2033\u2034\u2035\u2036\u2037\u2038\u2039\u203a\u203b\u203c\u203d\u203e\u203f\u2040\u2041\u2042\u2043\u2044\u2045\u2046\u2047\u2048\u2049\u204a\u204b\u204c\u204d\u204e\u204f\u2050\u2051\u2052\u2053\u2054\u2055\u2056\u2057\u2058\u2059\u205a\u205b\u205c\u205d\u205e\u205f\u2060\u2061\u2062\u2063\u2064\u2065\u2066\u2067\u2068\u2069\u206a\u206b\u206c\u206d\u206e\u206f\u2070\u2071\u2072\u2073\u2074\u2075\u2076\u2077\u2078\u2079\u207a\u207b\u207c\u207d\u207e\u207f\u2080\u2081\u2082\u2083\u2084\u2085\u2086\u2087\u2088\u2089\u208a\u208b\u208c\u208d\u208e\u208f\u2090\u2091\u2092\u2093\u2094\u2095\u2096\u2097\u2098\u2099\u209a\u209b\u209c\u209d\u209e\u209f\u20a0\u20a1\u20a2\u20a3\u20a4\u20a5\u20a6\u20a7\u20a8\u20a9\u20aa\u20ab\u20ac\u20ad\u20ae\u20af\u20b0\u20b1\u20b2\u20b3\u20b4\u20b5\u20b6\u20b7\u20b8\u20b9\u20ba\u20bb\u20bc\u20bd\u20be\u20bf\u20c0\u20c1\u20c2\u20c3\u20c4\u20c5\u20c6\u20c7\u20c8\u20c9\u20ca\u20cb\u20cc\u20cd\u20ce\u20cf\u20d0\u20d1\u20d2\u20d3\u20d4\u20d5\u20d6\u20d7\u20d8\u20d9\u20da\u20db\u20dc\u20dd\u20de\u20df\u20e0\u20e1\u20e2\u20e3\u20e4\u20e5\u20e6\u20e7\u20e8\u20e9\u20ea\u20eb\u20ec\u20ed\u20ee\u20ef\u20f0\u20f1\u20f2\u20f3\u20f4\u20f5\u20f6\u20f7\u20f8\u20f9\u20fa\u20fb\u20fc\u20fd\u20fe\u20ff\u2100\u2101\u2102\u2103\u2104\u2105\u2106\u2107\u2108\u2109\u210a\u210b\u210c\u210d\u210e\u210f\u2110\u2111\u2112\u2113\u2114\u2115\u2116\u2117\u2118\u2119\u211a\u211b\u211c\u211d\u211e\u211f\u2120\u2121\u2122\u2123\u2124\u2125\u2126\u2127\u2128\u2129\u212a\u212b\u212c\u212d\u212e\u212f\u2130\u2131\u2132\u2133\u2134\u2135\u2136\u2137\u2138\u2139\u213a\u213b\u213c\u213d\u213e\u213f\u2140\u2141\u2142\u2143\u2144\u2145\u2146\u2147\u2148\u2149\u214a\u214b\u214c\u214d\u214e\u214f\u2150\u2151\u2152\u2153\u2154\u2155\u2156\u2157\u2158\u2159\u215a\u215b\u215c\u215d\u215e\u215f\u2160\u2161\u2162\u2163\u2164\u2165\u2166\u2167\u2168\u2169\u216a\u216b\u216c\u216d\u216e\u216f\u2170\u2171\u2172\u2173\u2174\u2175\u2176\u2177\u2178\u2179\u217a\u217b\u217c\u217d\u217e\u217f\u2180\u2181\u2182\u2183\u2184\u2185\u2186\u2187\u2188\u2189\u218a\u218b\u218c\u218d\u218e\u218f\u2190\u2191\u2192\u2193\u2194\u2195\u2196\u2197\u2198\u2199\u219a\u219b\u219c\u219d\u219e\u219f\u21a0\u21a1\u21a2\u21a3\u21a4\u21a5\u21a6\u21a7\u21a8\u21a9\u21aa\u21ab\u21ac\u21ad\u21ae\u21af\u21b0\u21b1\u21b2\u21b3\u21b4\u21b5\u21b6\u21b7\u21b8\u21b9\u21ba\u21bb\u21bc\u21bd\u21be\u21bf\u21c0\u21c1\u21c2\u21c3\u21c4\u21c5\u21c6\u21c7\u21c8\u21c9\u21ca\u21cb\u21cc\u21cd\u21ce\u21cf\u21d0\u21d1\u21d2\u21d3\u21d4\u21d5\u21d6\u21d7\u21d8\u21d9\u21da\u21db\u21dc\u21dd\u21de\u21df\u21e0\u21e1\u21e2\u21e3\u21e4\u21e5\u21e6\u21e7\u21e8\u21e9\u21ea\u21eb\u21ec\u21ed\u21ee\u21ef\u21f0\u21f1\u21f2\u21f3\u21f4\u21f5\u21f6\u21f7\u21f8\u21f9\u21fa\u21fb\u21fc\u21fd\u21fe\u21ff\u21ff\u2200\u2201\u2202\u2203\u2204\u2205\u2206\u2207\u2208\u2209\u220a\u220b\u220c\u220d\u220e\u220f\u2210\u2211\u2212\u2213\u2214\u2215\u2216\u2217\u2218\u2219\u221a\u221b\u221c\u221d\u221e\u221f\u2220\u2221\u2222\u2223\u2224\u2225\u2226\u2227\u2228\u2229\u222a\u222b\u222c\u222d\u222e\u222f\u2230\u2231\u2232\u2233\u2234\u2235\u2236\u2237\u2238\u2239\u223a\u223b\u223c\u223d\u223e\u223f\u2240\u2241\u2242\u2243\u2244\u2245\u2246\u2247\u2248\u2249\u224a\u224b\u224c\u224d\u224e\u224f\u2250\u2251\u2252\u2253\u2254\u2255\u2256\u2257\u2258\u2259\u225a\u225b\u225c\u225d\u225e\u225f\u2260\u2261\u2262\u2263\u2264\u2265\u2266\u2267\u2268\u2269\u226a\u226b\u226c\u226d\u226e\u226f\u2270\u2271\u2272\u2273\u2274\u2275\u2276\u2277\u2278\u2279\u227a\u227b\u227c\u227d\u227e\u227f\u2280\u2281\u2282\u2283\u2284\u2285\u2286\u2287\u2288\u2289\u228a\u228b\u228c\u228d\u228e\u228f\u2290\u2291\u2292\u2293\u2294\u2295\u2296\u2297\u2298\u2299\u229a\u229b\u229c\u229d\u229e\u229f\u22a0\u22a1\u22a2\u22a3\u22a4\u22a5\u22a6\u22a7\u22a8\u22a9\u22aa\u22ab\u22ac\u22ad\u22ae\u22af\u22b0\u22b1\u22b2\u22b3\u22b4\u22b5\u22b6\u22b7\u22b8\u22b9\u22ba\u22bb\u22bc\u22bd\u22be\u22bf\u22c0\u22c1\u22c2\u22c3\u22c4\u22c5\u22c6\u22c7\u22c8\u22c9\u22ca\u22cb\u22cc\u22cd\u22ce\u22cf\u22d0\u22d1\u22d2\u22d3\u22d4\u22d5\u22d6\u22d7\u22d8\u22d9\u22da\u22db\u22dc\u22dd\u22de\u22df\u22e0\u22e1\u22e2\u22e3\u22e4\u22e5\u22e6\u22e7\u22e8\u22e9\u22ea\u22eb\u22ec\u22ed\u22ee\u22ef\u22f0\u22f1\u22f2\u22f3\u22f4\u22f5\u22f6\u22f7\u22f8\u22f9\u22fa\u22fb\u22fc\u22fd\u22fe\u22ff\u22ff\u2300\u2301\u2302\u2303\u2304\u2305\u2306\u2307\u2308\u2309\u230a\u230b\u230c\u230d\u230e\u230f\u2310\u2311\u2312\u2313\u2314\u2315\u2316\u2317\u2318\u2319\u231a\u231b\u231c\u231d\u231e\u231f\u2320\u2321\u2322\u2323\u2324\u2325\u2326\u2327\u2328\u2329\u232a\u232b\u232c\u232d\u232e\u232f\u2330\u2331\u2332\u2333\u2334\u2335\u2336\u2337\u2338\u2339\u233a\u233b\u233c\u233d\u233e\u233f\u2340\u2341\u2342\u2343\u2344\u2345\u2346\u2347\u2348\u2349\u234a\u23
```

2.2.3 Feature Engineering

Feature engineering adalah proses pembuatan fitur-fitur baru dari data mentah yang dapat memberikan insight tambahan untuk analisis. Pada tahap ini, berbagai metrik kuantitatif dihitung dari setiap tweet untuk mendukung analisis yang lebih mendalam. Fitur-fitur ini mencakup statistik teks dasar, metrik engagement yang dinormalisasi, dan skor komposit yang mengindikasikan potensi viralitas dan kredibilitas sumber.

Fitur	Deskripsi	Formula
char_count	Jumlah karakter	len(text)
word_count	Jumlah kata	len(text.split())
hashtag_count	Jumlah hashtag	Regex extraction
mention_count	Jumlah mention	Regex extraction
emoji_count	Jumlah emoji	Unicode range detection
engagement_rate	Tingkat engagement	like_count / max(user_followers, 1)
virality_score	Skor viralitas	retweet_count + like_count
user_credibility_score	Kredibilitas user	(followers/1M)*0.5 + (verified)*0.5

2.3 Quality Gate (Validasi Kualitas Data)

Quality Gate berfungsi sebagai checkpoint yang memvalidasi kualitas dataset sebelum digunakan untuk training model. Komponen ini mengimplementasikan serangkaian pemeriksaan otomatis yang memastikan data memenuhi standar minimum untuk menghasilkan model yang reliable. Jika dataset tidak memenuhi threshold yang ditetapkan, proses training akan dihentikan dan sistem akan menunggu hingga tersedia data yang cukup berkualitas.

2.3.1 Quality Thresholds Threshold kualitas didefinisikan berdasarkan pengalaman empiris dan best practice dalam pengembangan model topic modeling. Nilai-nilai

threshold ini dapat dikonfigurasi melalui environment variable untuk memungkinkan penyesuaian sesuai dengan karakteristik data yang berbeda.

```
# Quality thresholds
min_dataset_size = 10          # Minimum jumlah tweet untuk training
min_unique_users = 5           # Minimum user unik
max_duplicate_ratio = 0.1      # Maximum rasio duplikasi (10%)
min_avg_quality_score = 0.3    # Minimum skor kualitas
max_error_rate = 0.05         # Maximum error rate (5%)
```

2.3.2 Quality Score Calculation Perhitungan skor kualitas menggunakan pendekatan weighted scoring yang mempertimbangkan empat dimensi utama: ukuran dataset, keragaman sumber data, kualitas konten, dan tingkat engagement. Setiap dimensi memiliki bobot yang mencerminkan tingkat kepentingannya terhadap kualitas model yang dihasilkan. Skor akhir merupakan kombinasi tertimbang dari keempat komponen tersebut dan digunakan untuk menentukan apakah dataset layak untuk training.

```
weights = {
    'size': 0.3,          # 30% - Ukuran dataset
    'diversity': 0.2,      # 20% - Keragaman user
    'content': 0.3,        # 30% - Kualitas konten
    'engagement': 0.2,     # 20% - Tingkat engagement
}

# Size score
size_score = min(1.0, total_tweets / min_dataset_size)

# Diversity score
diversity_score = min(1.0, unique_users / min_unique_users)

# Content quality score
content_score = 0.5 if avg_length >= 50 else 0.3 if avg_length >= 30 else 0
content_score += 0.5 if too_short_ratio < 0.1 else 0.3 if too_short_ratio < 0.2 else 0

# Engagement score
engagement_score = min(1.0, avg_engagement_rate * 100)
```

2.3.3 Anomaly Detection Selain validasi kualitas dasar, sistem juga melakukan deteksi anomali untuk mengidentifikasi pola-pola tidak normal dalam dataset. Anomali ini dapat mengindikasikan masalah pada proses pengumpulan data, perubahan behavior platform Twitter, atau adanya aktivitas spam/bot. Setiap anomali yang terdeteksi dicatat dengan severity level yang sesuai untuk membantu tim dalam melakukan investigasi dan tindakan korektif.

Anomaly Type	Threshold	Severity
Low Engagement	avg_likes < 1 AND avg_retweets < 1	Warning

Anomaly Type	Threshold	Severity
High URL Ratio	url_ratio > 80%	Warning (spam indicator)
Language Diversity	languages > 5	Info
Short Content	avg_length < 30 chars	Warning

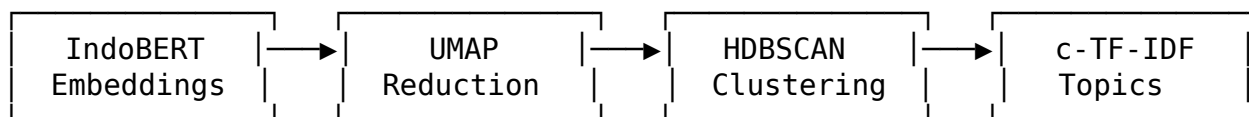
3. Proses Eksperimen Training

Bab ini menjelaskan proses eksperimen training yang dilakukan untuk membangun model topic modeling. Eksperimen training meliputi pemilihan arsitektur model, konfigurasi hyperparameter, implementasi pipeline training, dan proses fine-tuning. Dokumentasi yang detail mengenai eksperimen ini penting untuk memastikan reproducibility dan memungkinkan iterasi pengembangan yang sistematis.

3.1 Arsitektur Model

Arsitektur model yang dipilih untuk proyek ini adalah BERTopic, sebuah teknik topic modeling state-of-the-art yang menggabungkan kekuatan neural embeddings dengan algoritma clustering tradisional. Pemilihan BERTopic didasarkan pada kemampuannya menghasilkan topik yang lebih koheren dan interpretable dibandingkan metode tradisional seperti LDA (Latent Dirichlet Allocation).

3.1.1 BERTopic Overview BERTopic merupakan framework topic modeling modern yang dikembangkan oleh Maarten Grootendorst. Berbeda dengan metode tradisional yang mengandalkan bag-of-words representation, BERTopic memanfaatkan contextual embeddings dari model transformer untuk menangkap makna semantik yang lebih kaya dari dokumen. Proses BERTopic terdiri dari empat tahap utama yang saling terintegrasi: 1. **Document Embeddings** menggunakan Transformer 2. **Dimensionality Reduction** dengan UMAP 3. **Clustering** dengan HDBSCAN 4. **Topic Representation** dengan c-TF-IDF



3.1.2 Konfigurasi Model Konfigurasi model ditetapkan berdasarkan hasil eksperimen awal dan karakteristik data tweet berbahasa Indonesia. Parameter utama yang perlu diperhatikan adalah pemilihan embedding model yang sesuai dengan bahasa target dan ukuran minimum cluster yang disesuaikan dengan volume data yang tersedia. Konfigurasi ini dapat dimodifikasi melalui environment variable untuk memudahkan eksperimen dengan parameter yang berbeda.

```
# Embedding model
embedding_model_name = "indobenchmark/indobert-base-p1"
```

```

# BERTopic parameters
min_topic_size = 2           # Minimum dokumen per topik
nr_topics = "auto"          # Otomatis menentukan jumlah topik
calculate_probabilities = True

# Custom vectorizer
vectorizer_model = CountVectorizer(
    ngram_range=(1, 2),      # Unigram dan bigram
    stop_words=None,         # Semua kata (multilingual)
    min_df=2                 # Minimum document frequency
)

```

3.2 Training Pipeline

Training pipeline mengimplementasikan end-to-end workflow dari loading data hingga model evaluation. Pipeline ini dirancang untuk berjalan secara otomatis sebagai bagian dari scheduled job, namun juga dapat dieksekusi secara manual untuk keperluan eksperimen. Setiap tahap dalam pipeline dilengkapi dengan logging yang comprehensive untuk memudahkan debugging dan monitoring.

3.2.1 Data Loading Tahap data loading mengambil data training dari database PostgreSQL dengan filter berdasarkan waktu (default: 7 hari terakhir). Query dirancang untuk mengambil hanya tweet yang sudah melalui proses preprocessing dan memenuhi kriteria kualitas minimum. Penggunaan window waktu yang bergerak memastikan model selalu dilatih dengan data yang relevan dan up-to-date.

```

def get_training_data(hours: int = 168) -> pd.DataFrame:
    """Get training data from database (last 7 days)."""
    query = """
        SELECT tweet_id, text, created_at, user_id,
               like_count, retweet_count, engagement_rate, lang
        FROM tweets
        WHERE processed_at > %s
              AND char_count >= 20
              AND lang IN ('id', 'in', 'en')
        ORDER BY created_at DESC
    """
    return pd.DataFrame(db.fetch_dict(query, (cutoff_time,)))

```

3.2.2 Data Preparation Setelah data dimuat dari database, tahap data preparation melakukan transformasi final untuk menyiapkan data dalam format yang dibutuhkan oleh BERTopic. Proses ini mencakup filtering tambahan berdasarkan panjang teks dan konversi dataframe menjadi list of strings yang dapat langsung diproses oleh model embedding.

```

def prepare_data(df: pd.DataFrame) -> List[str]:
    """Prepare texts for training."""
    # Filter short texts

```



```
df = df[df['text'].str.len() >= 20].copy()

# Return list of texts
return df['text'].tolist()
```

3.2.3 Model Training Proses training model merupakan inti dari pipeline yang menggabungkan embedding generation, dimensionality reduction, clustering, dan topic representation dalam satu workflow terintegrasi. Model IndoBERT yang digunakan sebagai embedding model telah dilatih secara khusus untuk memahami konteks bahasa Indonesia, sehingga menghasilkan representasi vektor yang lebih akurat untuk teks berbahasa Indonesia.

```
def train_model(texts: List[str]) -> BERTopic:
    """Train BERTopic model."""
    # Initialize embedding model (IndoBERT)
    embedding_model = SentenceTransformer("indobenchmark/indobert-base-p1")

    # Initialize BERTopic
    topic_model = BERTopic(
        embedding_model=embedding_model,
        vectorizer_model=vectorizer_model,
        min_topic_size=2,
        nr_topics="auto",
        calculate_probabilities=True,
        verbose=True
    )

    # Fit model
    topics, probs = topic_model.fit_transform(texts)

    return topic_model
```

3.3 Hyperparameter Tuning

Hyperparameter tuning merupakan proses penting untuk mengoptimalkan performa model sesuai dengan karakteristik data yang dihadapi. Berbeda dengan supervised learning yang memiliki metrik objektif seperti akurasi, topic modeling memerlukan evaluasi yang lebih nuanced yang mempertimbangkan interpretability dan coherence dari topik yang dihasilkan.

3.3.1 Parameter Space Parameter space yang dieksplorasi dalam proses tuning mencakup konfigurasi BERTopic, vectorizer, dan embedding model. Tabel berikut menunjukkan range nilai yang dipertimbangkan untuk setiap parameter beserta nilai default yang digunakan dalam production:

Parameter	Default	Range	Description
min_topic_size	2	2-50	Minimum documents per topic
nr_topics	auto	auto/5-50	Number of topics
ngram_range	(1,2)	(1,1)-(1,3)	N-gram range for vectorizer
min_df	2	1-10	Minimum document frequency

3.3.2 Embedding Model Selection Pemilihan embedding model merupakan keputusan paling kritis dalam pipeline BERTopic karena kualitas embedding secara langsung mempengaruhi kemampuan model untuk menangkap similarity semantik antar dokumen. Beberapa kandidat model dievaluasi berdasarkan dukungan bahasa, ukuran model, dan performa empiris pada dataset tweet berbahasa Indonesia.

Model	Language	Size	Performance
indobenchmark/indobert-base-p1	Indonesian	124M	✓ Selected
paraphrase-multilingual-MiniLM-L12-v2	Multilingual	118M	Alternative
sentence-transformers/LaBSE	Multilingual	471M	Heavy

Alasan pemilihan IndoBERT: - Dilatih khusus untuk Bahasa Indonesia - Performa terbaik untuk teks berbahasa Indonesia - Ukuran model reasonable untuk production

4. Evaluasi Model

Evaluasi model topic modeling memiliki tantangan tersendiri karena tidak adanya ground truth labels seperti pada supervised learning. Bab ini menjelaskan berbagai metrik yang digunakan untuk mengukur kualitas topik yang dihasilkan, proses deteksi drift untuk monitoring model di production, serta contoh output evaluasi dari eksperimen yang telah dilakukan.

4.1 Metrik Evaluasi

Metrik evaluasi yang digunakan dalam proyek ini dirancang untuk mengukur berbagai aspek kualitas model topic modeling. Metrik-metrik ini memberikan gambaran kuantitatif mengenai jumlah topik yang ditemukan, distribusi dokumen antar topik, dan proporsi dokumen yang tidak dapat diklasifikasikan (outliers).

4.1.1 Topic Quality Metrics Topic quality metrics mengukur karakteristik statistik dari topik-topik yang dihasilkan model. Metrik-metrik ini memberikan insight mengenai granularity topik, coverage dokumen, dan keseimbangan distribusi. Implementasi fungsi evaluasi berikut menunjukkan perhitungan berbagai metrik kualitas dari model BERTopic:

```
def evaluate_model(topic_model: BERTopic, texts: List[str]) -> Dict[str, Any]:
    """Evaluate model quality."""
    topic_info = topic_model.get_topic_info()
```

```

metrics = {
    # Number of discovered topics (excluding outliers)
    'num_topics': len(topic_info) - 1,

    # Average documents per topic
    'avg_topic_size': topic_info['Count'].mean(),

    # Total documents processed
    'total_documents': len(texts),

    # Ratio of outlier documents (topic = -1)
    'outliers_ratio': np.sum(topics_pred == -1) / len(topics_pred),

    # Gini coefficient for topic balance
    # 0 = perfect balance, 1 = all in one topic
    'topic_balance_gini': calculate_gini(topic_counts),
}

return metrics

```

4.1.2 Interpretasi Metrik Untuk menginterpretasikan hasil evaluasi model, diperlukan pemahaman mengenai makna dan nilai ideal dari setiap metrik. Tabel berikut memberikan panduan interpretasi yang dapat digunakan untuk menilai apakah model yang dihasilkan memenuhi standar kualitas yang diharapkan:

Metrik	Nilai Ideal	Interpretasi
num_topics	5-20	Jumlah topik yang bermakna
avg_topic_size	> 10	Rata-rata dokumen per topik
outliers_ratio	< 0.3	Maksimal 30% dokumen tidak terklasifikasi
topic_balance_gini	< 0.5	Distribusi topik yang seimbang

4.2 Topic Drift Detection

Topic drift detection merupakan mekanisme monitoring yang penting untuk mendeteksi perubahan signifikan pada topik-topik yang ditemukan model dari waktu ke waktu. Drift dapat terjadi karena perubahan tren diskusi di media sosial, event tertentu yang mengubah landscape percakapan, atau perubahan karakteristik data input. Deteksi drift yang akurat memungkinkan tim untuk mengambil tindakan proaktif seperti re-training atau investigasi lebih lanjut.

4.2.1 Algoritma Drift Detection Algoritma drift detection yang diimplementasikan menggunakan Jaccard similarity untuk membandingkan kata-kata kunci (top words) antara topik model saat ini dengan model sebelumnya. Pendekatan ini efektif karena perubahan pada top words mencerminkan perubahan substansial pada makna topik.

```

def detect_topic_drift(current_topics, previous_run_id) -> Dict[str, Any]:
    """
    Detect topic drift compared to previous model.
    Uses Jaccard similarity of top words per topic.
    """
    def get_top_words(topic_list, n=10):
        return set([word for word, _ in topic_list[:n]])

    similarities = []
    for topic_id, words in current_topics.items():
        current_words = get_top_words(words)

        # Find most similar previous topic
        max_similarity = 0
        for prev_words in previous_topics.values():
            prev_words_set = set([w for w, _ in prev_words[:10]])
            # Jaccard similarity
            similarity = len(current_words & prev_words_set) / len(current_words | prev_words_set)
            max_similarity = max(max_similarity, similarity)

        similarities.append(max_similarity)

    avg_similarity = np.mean(similarities)
    drift_score = 1 - avg_similarity # 0 = no drift, 1 = complete drift

    return {
        'drift_detected': drift_score > 0.5, # Threshold
        'drift_score': drift_score,
        'avg_topic_similarity': avg_similarity,
    }

```

4.2.2 Drift Threshold Threshold untuk menentukan tingkat keparahan drift ditetapkan berdasarkan analisis empiris terhadap variasi normal antar training runs. Tabel berikut mendefinisikan interpretasi dan recommended action untuk setiap range drift score:

Drift Score	Interpretasi	Action
0.0 - 0.2	No drift	Continue monitoring
0.2 - 0.5	Minor drift	Log warning
0.5 - 1.0	Significant drift	Alert & investigate

4.3 Contoh Output Evaluasi

Berikut adalah contoh output evaluasi dari salah satu training run yang menunjukkan model dengan kualitas yang baik. Output ini tersimpan di MLflow dan dapat diakses untuk keperluan audit dan comparison antar eksperimen:

```
{
  "num_topics": 8,
  "avg_topic_size": 15.5,
  "total_documents": 124,
  "outliers_ratio": 0.15,
  "topic_balance_gini": 0.35,
  "drift_detected": false,
  "drift_score": 0.12
}
```

5. MLOps Tooling & Reproducibility

Reproducibility (kemampuan untuk mereproduksi hasil eksperimen) merupakan prinsip fundamental dalam pengembangan machine learning yang serius. Bab ini menjelaskan berbagai tools dan praktik yang diimplementasikan untuk memastikan setiap eksperimen dapat dilacak, diaudit, dan direproduksi. Infrastruktur MLOps yang dibangun mencakup experiment tracking, artifact storage, model versioning, dan automated scheduling.

5.1 MLflow Integration

MLflow dipilih sebagai platform utama untuk experiment tracking dan model management karena menyediakan solusi end-to-end yang terintegrasi dengan baik dengan ekosistem Python. MLflow memungkinkan pencatatan otomatis untuk parameter, metrik, dan artefak dari setiap training run, serta menyediakan UI web untuk visualisasi dan comparison eksperimen.

5.1.1 Experiment Tracking Experiment tracking menggunakan MLflow memungkinkan pencatatan komprehensif untuk setiap training run. Informasi yang dicatat mencakup parameter model, metrik evaluasi, dan artefak seperti file model dan visualisasi. Setiap run mendapatkan unique identifier yang memungkinkan traceability lengkap dari hasil model kembali ke konfigurasi dan data yang digunakan.

Setup MLflow

```
mlflow.set_tracking_uri("http://mlflow:5000")
mlflow.set_experiment("bertopic-pemerintah")
```

with mlflow.start_run() **as** run:

Log parameters

```
mlflow.log_param("embedding_model", "indobenchmark/indobert-base-pl")
mlflow.log_param("min_topic_size", 2)
mlflow.log_param("num_documents", len(texts))
mlflow.log_param("nr_topics", "auto")
```

Log metrics

```
mlflow.log_metrics({
```

```

    'num_topics': eval_metrics['num_topics'],
    'avg_topic_size': eval_metrics['avg_topic_size'],
    'outliers_ratio': eval_metrics['outliers_ratio'],
    'topic_balance_gini': eval_metrics['topic_balance_gini'],
    'drift_score': drift_info['drift_score'],
})

# Log artifacts
mlflow.log_artifact(topic_info_csv)      # Topic info table
mlflow.log_artifact(model_pickle)        # Serialized model

# Register model
mlflow.register_model(model_uri, "bertopic-pemerintah-model")

```

5.1.2 Model Registry MLflow Model Registry menyediakan centralized repository untuk mengelola lifecycle model dari development hingga production. Setiap versi model yang diregistrasi dapat diberi stage (None, Staging, Production, Archived) yang menunjukkan status deployment-nya. Fitur ini memungkinkan transisi model yang terkelola dan rollback yang mudah jika diperlukan.

MLflow Model Registry		
Model: bertopic-pemerintah-model		
Version 1	Stage: Production	Date: 2025-11-17
Version 2	Stage: Staging	Date: 2025-11-20
Version 3	Stage: None	Date: 2025-11-27

5.2 Artifact Storage (MinIO)

MinIO digunakan sebagai object storage yang S3-compatible untuk menyimpan berbagai artefak pipeline. Pemilihan MinIO didasarkan pada kemampuannya menyediakan storage yang scalable dengan API yang kompatibel dengan Amazon S3, memungkinkan migrasi ke cloud storage di masa depan tanpa perubahan kode yang signifikan.

5.2.1 Bucket Structure Struktur bucket dirancang untuk memisahkan data berdasarkan stage dalam pipeline dan memudahkan management lifecycle data. Konvensi penamaan menggunakan timestamp memungkinkan identifikasi mudah dan implementasi retention policy berbasis waktu.

```

mlops-data/
├── raw/                          # Raw tweets (JSONL)
│   └── tweets_20251117_*.jsonl
├── processed/                    # Processed tweets
│   └── tweets_20251117_*.jsonl
└── metadata/                     # Session metadata

```

```

└─ scraper_20251117_*.json
└─ models/                                # Model backups
    └─ bertopic_*.pkl

mlflow-artifacts/
└─ 1/                                     # Experiment ID
    └─ <run_id>/
        └─ artifacts/
            └─ topic_info.csv
                └─ model/
                    └─ bertopic_model.pkl
        └─ metrics/

```

5.3 Reproducibility Checklist

Checklist berikut merangkum praktik-praktik reproducibility yang telah diimplementasikan dalam proyek ini. Setiap aspek telah diverifikasi dan divalidasi untuk memastikan eksperimen dapat direproduksi dengan konsisten:

Aspek	Implementasi	Status
Version Control	Git repository	☐
Data Versioning	MinIO dengan timestamps	☐
Model Versioning	MLflow Model Registry	☐
Environment	Docker containers	☐
Dependencies	requirements.txt	☐
Config Management	Environment variables	☐
Experiment Tracking	MLflow	☐
Logging	Structured JSON logs	☐
Monitoring	Prometheus + Grafana	☐

5.4 Airflow DAG Scheduling

Apache Airflow digunakan sebagai orchestrator untuk menjadwalkan dan mengelola eksekusi pipeline secara otomatis. DAG (Directed Acyclic Graph) yang didefinisikan menentukan urutan eksekusi task dan dependency antar komponen pipeline. Scheduling menggunakan pendekatan “humanized” yang menyesuaikan waktu eksekusi dengan pola aktivitas pengguna Twitter di Indonesia.

5.4.1 Pipeline Schedule Jadwal pipeline dikonfigurasi untuk berjalan pada empat window waktu per hari yang dipilih berdasarkan analisis pola aktivitas Twitter. Setiap window merepresentasikan periode dengan volume dan engagement tweet yang signifikan:

```

# DAG Configuration
schedule_interval = '*/15 * * * *' # Every 15 minutes
max_active_runs = 1                 # Only one run at a time

```

```
# Activity Windows (4x per day)
WINDOWS = [
    {'name': 'morning', 'start_h': 7, 'start_m': 15},
    {'name': 'lunch', 'start_h': 12, 'start_m': 45},
    {'name': 'evening', 'start_h': 18, 'start_m': 20},
    {'name': 'night', 'start_h': 21, 'start_m': 30},
]
```

5.4.2 Window Enforcement Untuk mencegah duplikasi eksekusi yang tidak perlu dan mengoptimalkan penggunaan resource, sistem mengimplementasikan mekanisme window enforcement menggunakan Redis sebagai state store. Mekanisme ini memastikan bahwa hanya satu pipeline run yang berhasil dieksekusi per window waktu, meskipun DAG scheduler melakukan multiple trigger attempts.

```
window_key = f"scheduler:window:{date}:{window_name}"
# Check if already run
if redis.get(window_key) == 'success':
    return 'skip' # Skip this run
# After successful run
redis.set(window_key, 'success')
```

6. Kesimpulan dan Rekomendasi

Bab terakhir ini menyajikan ringkasan pencapaian proyek, analisis performa model, rekomendasi untuk pengembangan selanjutnya, dan lessons learned dari proses implementasi. Informasi ini diharapkan dapat menjadi referensi berharga untuk iterasi pengembangan berikutnya dan proyek-proyek serupa di masa depan.

6.1 Ringkasan Pencapaian

Proyek MLOps untuk analisis topik tweet telah berhasil mengimplementasikan pipeline end-to-end yang mencakup semua komponen yang direncanakan. Tabel berikut merangkum status implementasi untuk setiap komponen utama:

Komponen	Status	Catatan
Data Collection	☐ Complete	Anti-bot protection implemented
Data Preprocessing	☐ Complete	Validation & cleaning pipeline
Quality Gate	☐ Complete	Automated quality checks
Model Training	☐ Complete	BERTopic + IndoBERT
Experiment Tracking	☐ Complete	MLflow integration
Model Registry	☐ Complete	Version management
Drift Detection	☐ Complete	Jaccard similarity-based
Scheduling	☐ Complete	Airflow DAG (4x/day)

6.2 Performa Model

Berdasarkan hasil evaluasi dari multiple training runs, model BERTopic dengan IndoBERT embeddings menunjukkan performa yang konsisten dan stabil. Ringkasan performa berikut merepresentasikan karakteristik tipikal dari model yang dihasilkan oleh pipeline:

Model Performance Summary	
Embedding Model	: indobenchmark/indobert-base-p1
Topic Discovery	: 8-15 topics (auto-determined)
Outlier Ratio	: ~15% (acceptable)
Topic Balance	: Gini < 0.5 (well-balanced)
Training Time	: ~2-5 minutes (depends on data size)
Drift Score	: < 0.2 (stable over time)

6.3 Rekomendasi Pengembangan

Berdasarkan pengalaman implementasi dan analisis terhadap sistem yang berjalan, berikut adalah rekomendasi untuk pengembangan lebih lanjut. Rekomendasi dibagi menjadi dua kategori berdasarkan kompleksitas dan timeline implementasi.

6.3.1 Short-term Improvements Peningkatan jangka pendek yang dapat diimplementasikan dalam 1-3 bulan ke depan dengan effort yang relatif moderat: 1. **Sentiment Analysis**: Tambahkan analisis sentimen per topik 2. **Named Entity Recognition**: Ekstrak entitas (nama orang, organisasi, lokasi) 3. **Visualization Dashboard**: Dashboard interaktif untuk topic exploration

6.3.2 Long-term Improvements Peningkatan jangka panjang yang memerlukan perencanaan dan resource yang lebih signifikan, namun dapat memberikan value yang substantial:

1. **Active Learning**: Feedback loop untuk improve model quality
2. **Multi-modal Analysis**: Integrasi analisis gambar/media
3. **Real-time Processing**: Streaming pipeline dengan Kafka
4. **A/B Testing**: Framework untuk compare model versions

6.4 Lessons Learned

Berikut adalah pelajaran penting yang didapat selama proses pengembangan proyek ini. Insight ini dapat menjadi panduan untuk tim yang akan mengerjakan proyek serupa di masa depan:

1. **Data Quality is Key**: Quality gate prevents garbage-in-garbage-out
2. **Anti-bot Measures**: Essential for sustainable data collection
3. **MLflow for Everything**: Centralized tracking improves reproducibility
4. **Docker Everywhere**: Containerization ensures environment consistency

5. **Monitoring Early:** Prometheus/Grafana from day one catches issues fast

Lampiran

Lampiran berikut menyediakan informasi teknis tambahan yang dapat berguna untuk referensi dan troubleshooting.

A. Struktur Kode

Struktur direktori source code diorganisasi berdasarkan komponen fungsional pipeline. Setiap modul memiliki tanggung jawab spesifik dan dapat di-deploy secara independen sebagai container terpisah:

```
src/
├── scraper/main.py      # Twitter data collection
├── ingest/main.py      # Data preprocessing & storage
├── quality_gate/main.py # Quality validation
├── trainer/main.py     # BERTopic training
├── api/main.py         # REST API service
├── common/
│   ├── config.py      # Configuration management
│   ├── database.py    # PostgreSQL client
│   ├── storage.py     # MinIO client
│   ├── cache.py       # Redis client
│   ├── logging.py     # Structured logging
│   └── metrics.py     # Prometheus metrics
```

B. Environment Variables

Konfigurasi sistem dikelola melalui environment variables untuk memungkinkan deployment yang fleksibel di berbagai environment (development, staging, production). Berikut adalah daftar environment variables yang digunakan beserta keterangannya:

```
# Database
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
POSTGRES_USER=mlops
POSTGRES_PASSWORD=***
POSTGRES_DB=mlflow

# Storage
MINIO_ENDPOINT=minio:9000
MINIO_ACCESS_KEY=***
MINIO_SECRET_KEY=***

# MLflow
MLFLOW_TRACKING_URI=http://mlflow:5000
```

```
# Twitter
TWITTER_SEARCH_QUERY=pemerintah lang:id -filter:retweets
TWITTER_MAX_TWEETS=50
```

```
# Redis
REDIS_HOST=redis
REDIS_PORT=6379
```

C. Referensi

Daftar referensi akademis dan dokumentasi teknis yang digunakan dalam pengembangan proyek ini:

1. Grootendorst, M. (2022). BERTopic: Neural topic modeling with a class-based TF-IDF procedure. arXiv:2203.05794.
2. Wilie, B., et al. (2020). IndoNLU: Benchmark and Resources for Evaluating Indonesian Natural Language Understanding. AACL 2020.
3. MLflow Documentation: <https://mlflow.org/docs/latest/index.html>
4. Apache Airflow Documentation: <https://airflow.apache.org/docs/>

Dokumen ini dibuat sebagai bagian dari Tugas Proyek 4: Pelaporan Eksperimen dan Kualitas Model